

60

MINUTES

GITHUB COPILOT FUNDAMENTALS · QUICK WORKSHOP

GitHub Copilot Basic

既存コードの理解から始める 60分 Quick Workshop

Q0 最初的一步

Q1 コード理解

Q2 仕様・文書

Q3 リファクタリング

Q4 次の一步



受講前に使う環境とツールを確認する

01 / EDITOR

VS Code + Copilot Chat

Copilot Chatを利用できるプランでサインインします。

02 / SOURCE

Git

取得・ブランチ作業・差分確認に使用します。

03 / APP + TEST

Node.js 20.12.0+

`npm run app`と`npm test`に使用します。

Copilot Free / Student / Pro / Pro+ / Maxなど、Chatを利用できるプランが必要です。

Copilot Chatの応答 ✓ `node --version` ✓ `npm --version`

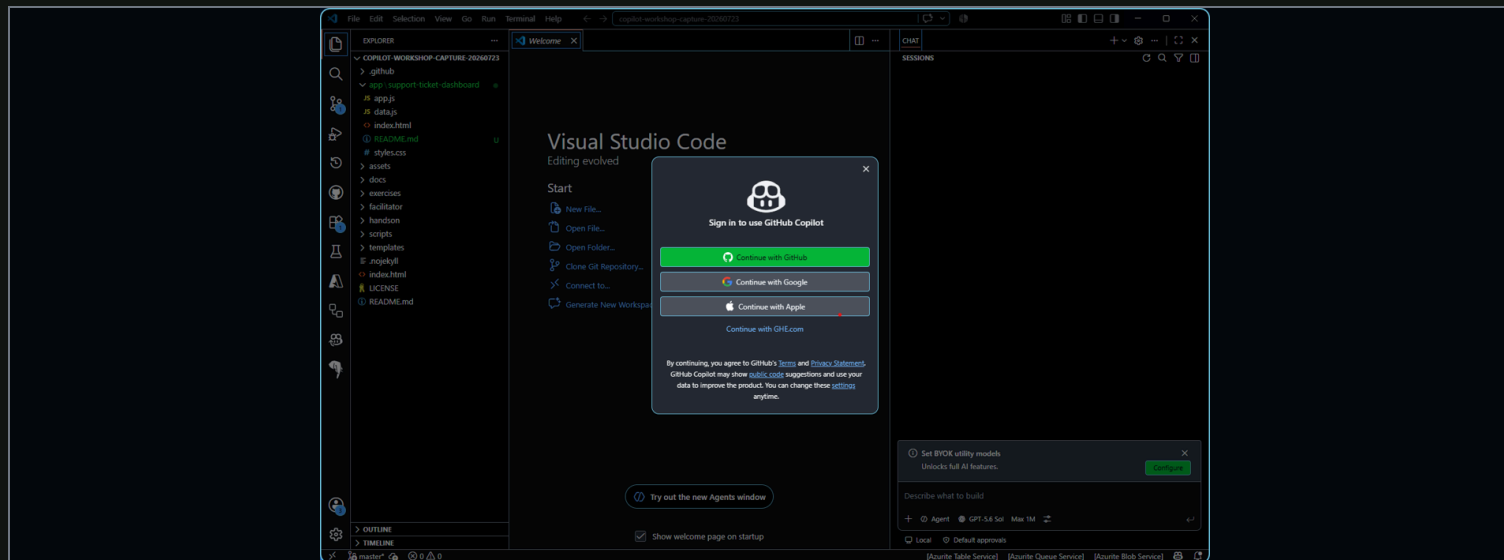


VS CodeからGitHub認証を始める

01 / VS CODE

Continue with GitHub

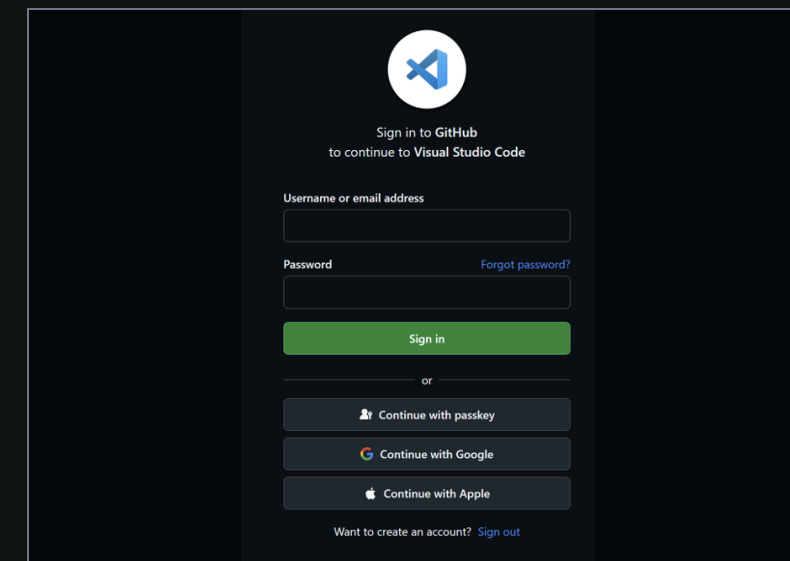
ステータスバーのCopilotマークを選び、GitHub認証を開始します。



02 / BROWSER

受講用アカウントでサインイン

Copilot Chatを利用できるGitHubアカウントを使います。



ブラウザーとVS Codeで別のGitHubアカウントを使わないでください。

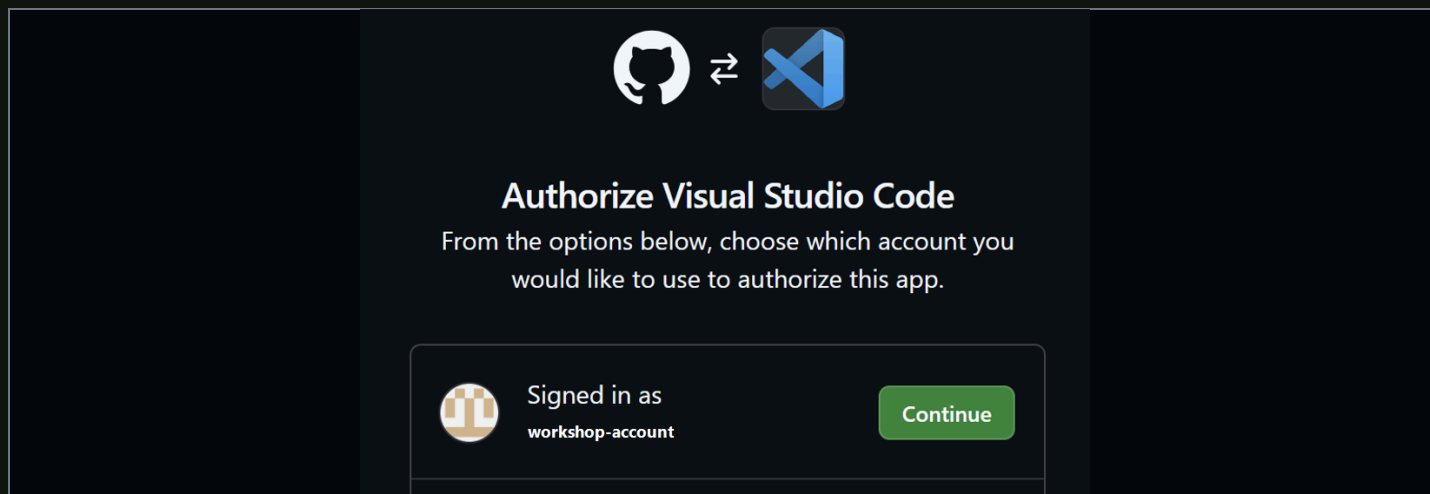


AuthorizeしてCopilot Chatを確認する

03 / AUTHORIZE

アカウントを確認してContinue

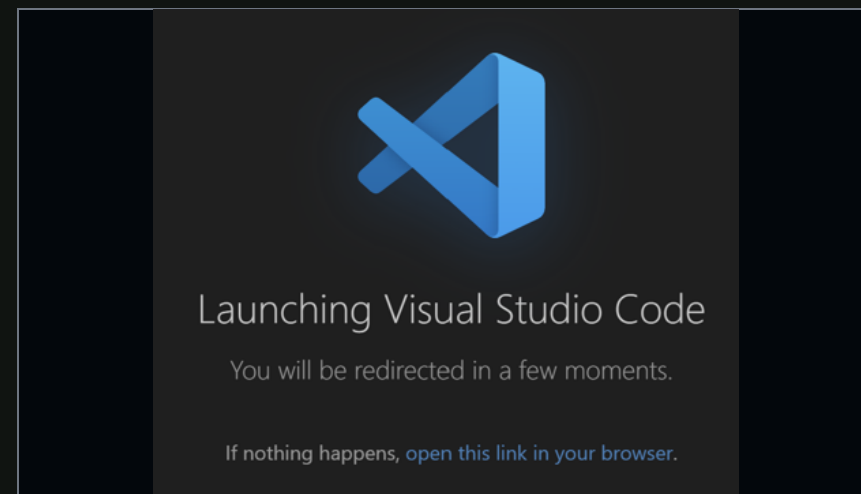
表示中のアカウントが受講用であることを確認します。



04 / RETURN

VS Codeへ戻る

ブラウザからVisual Studio Codeを開く操作を許可します。



05 / VERIFY

Copilot Chatへ簡単な質問を送り、応答を受け取れば準備完了



公開テンプレートから作成を始める

SOURCE TEMPLATE

Tachaan/workshop-github-copilot-basic-attendee

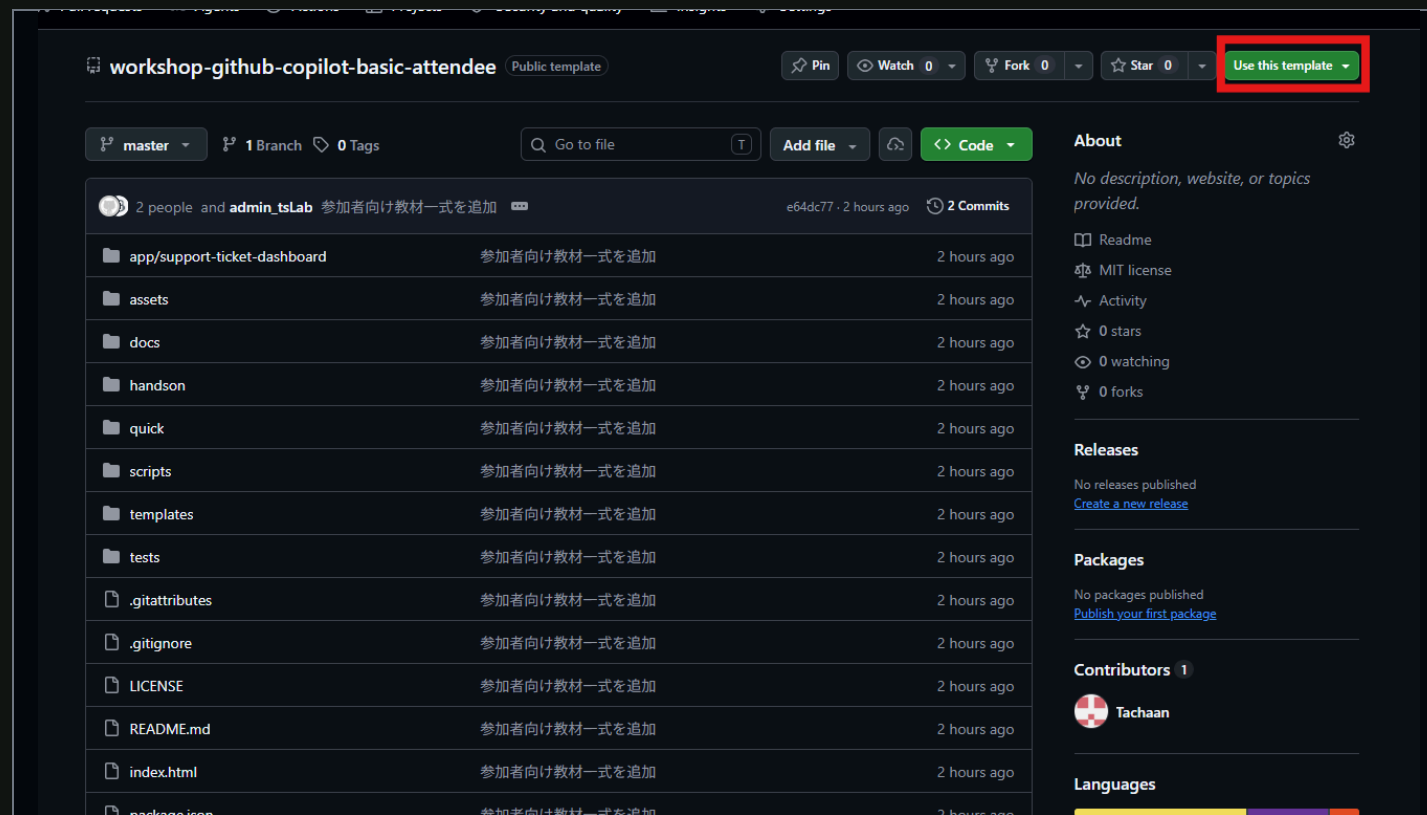
指定のPublic templateを開き、右上のボタンから作成画面へ進みます。

Use this template



Create a new repository

Forkではなく、Use this templateを使います。



自分の個人Owner配下に作成する

01 / OWNER	受講に使う自分の個人アカウント
02 / NAME	workshop-github-copilot-basic-workshop
03 / VISIBILITY	Privateを推奨
04 / COPILOT	JumpstartのPromptは空欄
05 / CREATE	Create repository

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

Start with a template

Templates pre-configure your repository with files.

Tachaan/workshop-github-copilot-basic-attendee

Include all branches

If enabled, all branches from the template repository will be included.

Off

1 General

Owner *
your account / workshop-github-copilot-basic-workshop
workshop-github-copilot-basic-workshop is available.

Description
Great repository names are short and memorable. How about [improved-fortnight?](#)
0 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository
Private

3 Jumpstart your project with Copilot (optional)

Tell [Copilot cloud agent](#) what you want to build in this repository. After creation, Copilot will open a pull request with generated files - such as a basic app, starter code, or other features you describe - then request your review when it's ready.

Prompt
Describe what you want Copilot to build
0 / 30000 characters

Create repository

Organizationへの作成は不要です。Include all branchesはOffのままにします。



今日のゴール

01 / UNDERSTAND

既存コードを理解する

Copilotの説明を、ファイル・関数・画面で照合する。

02 / ORGANIZE

仕様を整理する

事実、差分候補、要確認事項を分ける。

03 / CHANGE SAFELY

小さく安全に変更する

変更前後をテスト・差分・画面動作で確認する。

質問する

対象と目的を明示



照合する

実ファイルへ戻る



整理する

事実と判断を分離



小さく変える

範囲を固定



再確認する

テストと差分



60分のロードマップ

5 min	5 min	15 min	15 min	15 min	5 min
Overview	Q0	Q1	Q2	Q3	Q4
ゴール・環境・起動確認	最初の質問と回答の照合	既存コードを理解	仕様と実装の差分を整理	テストで守って小さく変更	振り返り

完璧な回答より、質問 → 照合 → 修正 → 再確認を一巡することを優先します。



教材サイトとDashboardは分けて開く

WEB MATERIAL

教材サイト

GitHub PagesでQ0-Q4を確認します。

[Quick版 Web教材](#)

WORKSHOP APP

Support Ticket Dashboard

公開テンプレートから作成した自分のリポジトリをローカルで起動します。

自分のRepository → clone → npm test → npm run app

```
# Local: repository root
npm run app
```

Dashboardに12件表示され、検索・絞り込みを操作できれば準備完了です。



Q0

GitHub Copilot利用の 最初の一歩

対象・目的・制約・出力形式を渡し、回答を実ファイルで確認します。



良い依頼は4点セット

TARGET

対象

`#file:../app.js`

PURPOSE

目的

初心者向けに役割を説明する

CONSTRAINT

制約

まだコードを変更しない

FORMAT

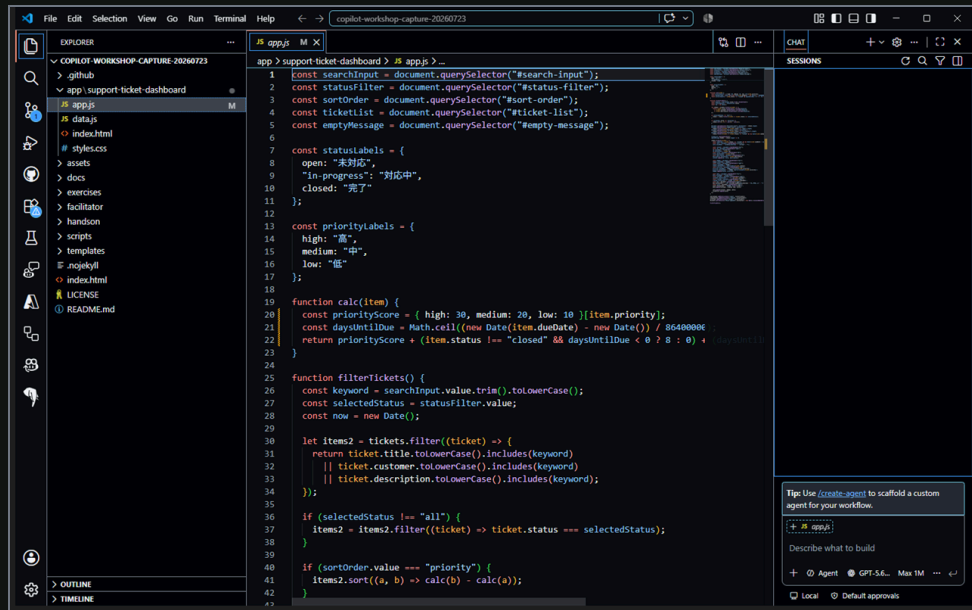
出力形式

入力・処理・出力の3項目

関数ごとの説明とコード上の根拠を求めると、確認可能な回答になりやすくなります。



最初のプロンプトを送る



Copilot Chatへ対象ファイルを追加

```
#file:app/support-ticket-dashboard/app.js
```

このファイルがSupport Ticket Dashboardで受け持つ役割を、初心者向けに1〜2文で説明してください。

1. 画面から受け取る入力
2. 主な処理
3. 画面へ返す出力

「主な処理」では、このファイル内でfunctionとして定義されている関数を見つけ、関数ごとに名前と役割を1〜2文で説明してください。

何を受け取り、何を処理し、結果をどこで使うかと、根拠となる変数名、関数名、DOM要素のIDを含めてください。まだコードは変更しないでください。



回答を実ファイルで照合する

- ファイル全体の役割が実際の処理と一致する
- 回答に挙げた関数が実在する
- 各関数の入力・処理・利用先がコードと一致する
- 検索・ステータス・並び順の変数とDOM IDが一致する

CHECKPOINT

コードで確かめる

関数ごとに入力・処理・利用先を確認する。

画面との接点になる変数とDOM IDを確認する。

回答の自然さではなく、コード上の根拠で判定します。



Q1

既存コードを Copilotで理解する

アプリの入口、主要ファイル、検索・絞り込み・描画の流れを追います。



まず4ファイルの責務をつなぐ

index.html

画面要素とJavaScriptの読み込み順

data.js

→ 12件のチケットと補助項目

app.js

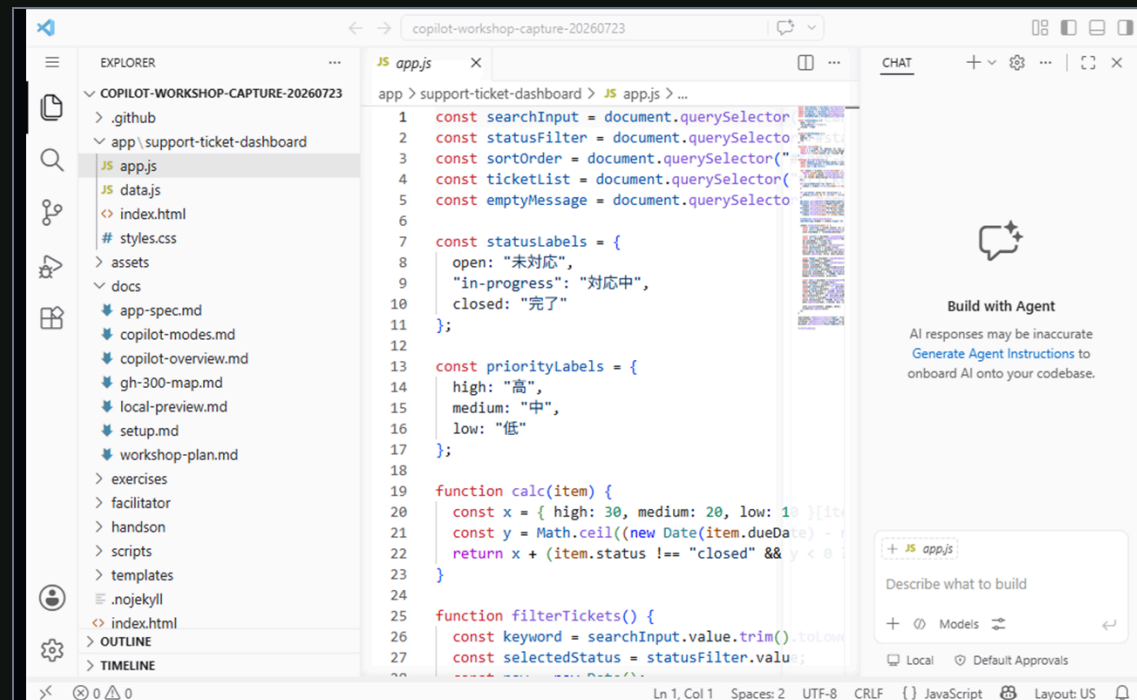
→ 検索、絞り込み、並び替え、描画

app-spec.md

→ 期待される画面と動作

問い

「どのファイルが、何を次へ渡すか？」



リポジトリ全体の関係を質問する

@workspace Support Ticket Dashboardが表示されるまでの流れを説明してください。

index.html、data.js、app.js、docs/app-spec.mdを対象に、「ファイル」「役割」「次に渡すデータ」の3列の表で回答してください。

internalMemoの具体値は引用・要約・出力せず、項目名と扱いだけを確認してください。

まだコードは変更せず、推測と確認できた事実を分けてください。

The screenshot shows a VS Code workspace named 'copilot-workshop-capture-20260723'. The file explorer on the left shows a project structure with files like 'index.html', 'data.js', 'app.js', and 'docs/app-spec.md'. The code editor in the center shows JavaScript code for a support ticket dashboard, including constants for search input, status filter, sort order, ticket list, and empty message, as well as functions for calculating priority and filtering tickets. The chat window on the right displays a table summarizing the relationships between files.

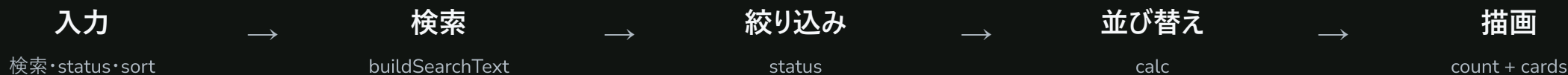
ファイル	役割	概観
data.js:1	検索・フィルタ対象となる [tickets] 配列を定義します。	先頭で [tickets] が定義され、HTML では [app.js] より先に読み込まれま す。
index.html:39-62	検索欄 [search-input]、ステータス [status-filter]、並び順 [sort- order]、空メッセージ [empty- message]、一覧 [ticket-list] を提供 します。	各IDが画面要素として定義されていま す。集計欄は [index.html:18-35]、スク リプトの読み込みは [index.html:65-66] です。
app.js:25-55	[filterTickets()] がキーワード検索、 ステータス絞り込み、並び替え、件数 更新を行います。	検索対象は [title]、[customer]、 [description] の3項目です。ステータ スは完全一致で絞り込み、優先度指定時 だけソートします。
app.js:19-23	[calc()] が優先度、期限切れ、期限 までの日数から並び替え用スコアを計 算します。	[sortOrder] が [priority] の場合、 算出スコアの降順で並び替えられます。
app.js:52-94	絞り込み結果を一覧へ再描画し、0件 表示や期限切れ表示を切り替えます。	[replaceChildren()] で一覧を空にした 後、結果ごとに [article] 要素を生成し ています。
app.js:97-102	画面操作と [filterTickets()] を接続 し、初期表示を実行します。	検索欄の [input]、各選択欄の [change] イベントに関数を登録し、最後に一度直 接呼び出しています。

InternalMemo は検索条件にも一覧描画にも参照されていないため、検索対象・画面表示のどちらにも含
まれません。具体値は引用・出力していません。コード変更も行っていないです。

回答に出たファイルを自分で開いて照合



filterTicketsを処理順に分解する



INLINE CHAT

app.jsの `filterTickets` 関数全体を選択してから実行

`/explain` この関数を、入力、検索、ステータス絞り込み、並び替え、件数更新、描画の順に説明してください。

検索では、入力値を検索用に整える処理と検索対象項目を明示してください。

各説明に対応するコードを示してください。

VERIFY

入力値の前処理、検索対象、絞り込み条件、並び替え、DOM更新を実コードで照合します。



Q2

仕様を整理し、 ドキュメント化する

仕様と実装を比べ、事実・差分候補・判断事項を分離します。



仕様と実装を同じ表で見る

SPEC

仕様書の記述

利用者が期待する画面・検索・ソート・表示項目。

IMPLEMENTATION

実装の根拠

ファイル、関数、条件式、DOM更新から確認できる事実。

DECISION

差分 / 要確認

どちらが正しいかは自動で決めず、担当者へ確認する。

見る観点: 検索対象 / ステータス表記 / ソート / 表示項目 / 期限切れ表示



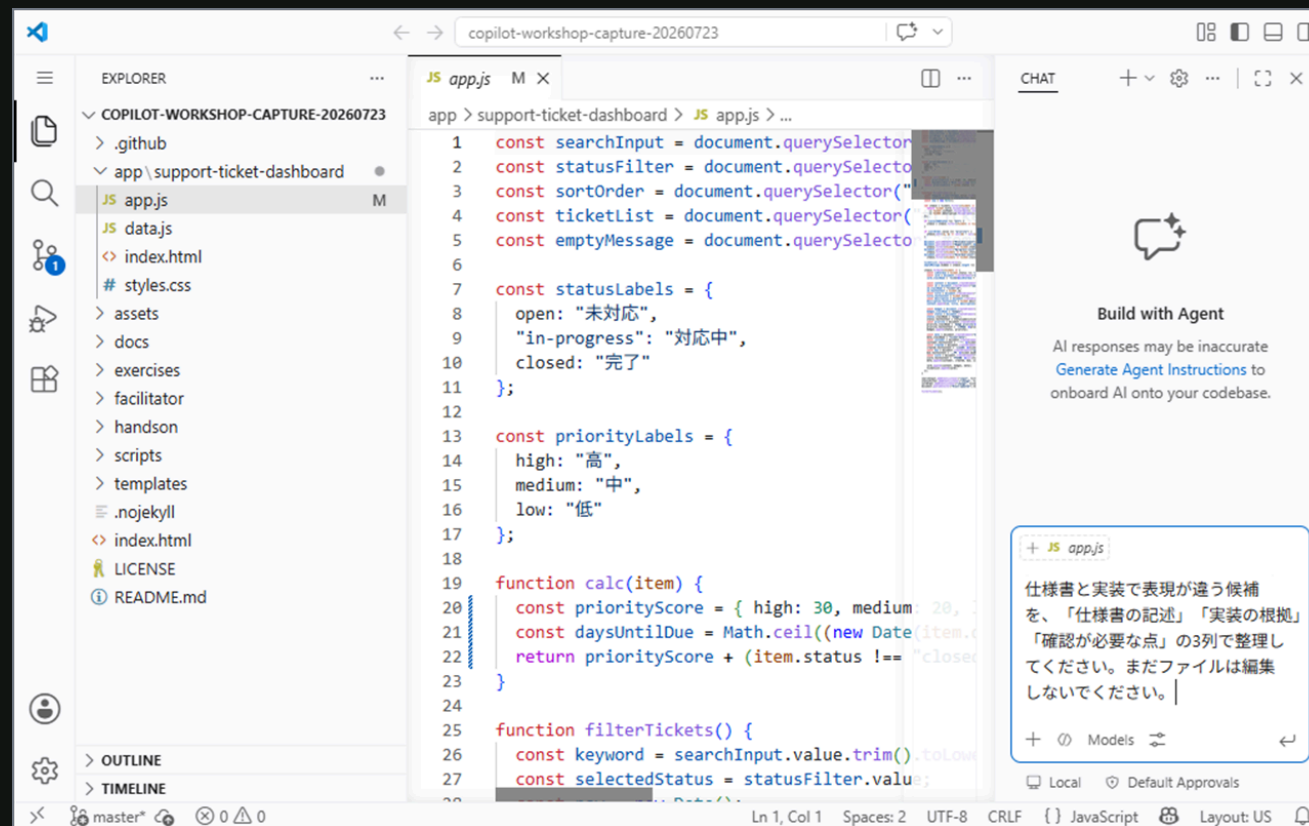
差分候補を3列で整理する

```
#file:docs/app-spec.md
#file:app/support-ticket-dashboard/app.js
```

仕様書と実装を比較してください。
 検索対象、ステータス表記、ソート、表示項目、
 期限切れ表示について、
 「仕様書の記述（行番号付き）」
 「実装の根拠（ファイル名、関数名、行番号付き）」
 「差分または要確認事項」の3列で整理してください。

internalMemoの具体値は引用・要約・出力せず、
 項目名と扱いだけを確認してください。

まだファイルは変更せず、実装から確認できないことは
 推測せず「要確認」としてください。



出力を3つのレーンへ分ける

CONFIRMED FACT

確認できた事実

コードや画面で根拠を示せる。文書へ記載できる候補。

DIFFERENCE

差分候補

仕様と実装が一致しない。どちらを直すかは未決定。

QUESTION

判断が必要

プロダクトオーナーや担当者へ確認する質問として残す。

internalMemo は具体値を出力しない。項目名と安全な扱いだけを文書化します。



Q3

既存コードを 安全にリファクタリングする

変更前の動作を確認し、検索・絞り込みだけを小さく分離します。



変更前の4操作を確認する

```
npm test
```

操作	期待結果
初期状態	12件
初期状態 → API 検索	1件
初期状態 → 対応中	4件
初期状態 → 一致しない語	0件 + 空表示

各行は独立。次の行へ進む前に、検索・ステータス・並び順を初期状態へ戻します。

CUSTOMER SUPPORT

Support Ticket Dashboard

最終更新: 2026/7/23

表示中
1

未対応
0

対応中
1

期限切れ
1

キーワード検索

ステータス

並び順

API

対応中

優先度の高い順

T-1008

対応中

優先度: 中

API の利用上限を確認したい

現在のプランで利用できる API 呼び出し回数を教えてください。

顧客: 西町テクノ

登録日: 2026-07-13

期限: 2026-07-22 (期限切れ)

スコア: 31

自動テスト + 手動確認の両方を残す



計画 → 適用 → 差分 → 再確認

1. PLAN FIRST

この関数の検索とステータス絞り込みを、純粋関数へ抽出する計画を作ってください。

並び替え、件数更新、DOM描画、`calc`の計算ルールは変更しません。

まず変更箇所、維持する挙動、確認手順だけを示し、まだ編集しないでください。

2. THEN APPLY

計画どおり、検索とステータス絞り込みだけを小さな関数へ抽出してください。

既存の命名とJavaScriptの書き方に合わせ、無関係な変更はしないでください。

SCOPE

変更してよい

検索とステータス絞り込みの小さな関数分割。

変更しない

検索対象、ソート、`calc`、件数、DOM描画、画面デザイン。

tests

npm test + 4操作



refactor

計画 → 適用



diff review

git diff



rerun tests

同じテストと4操作

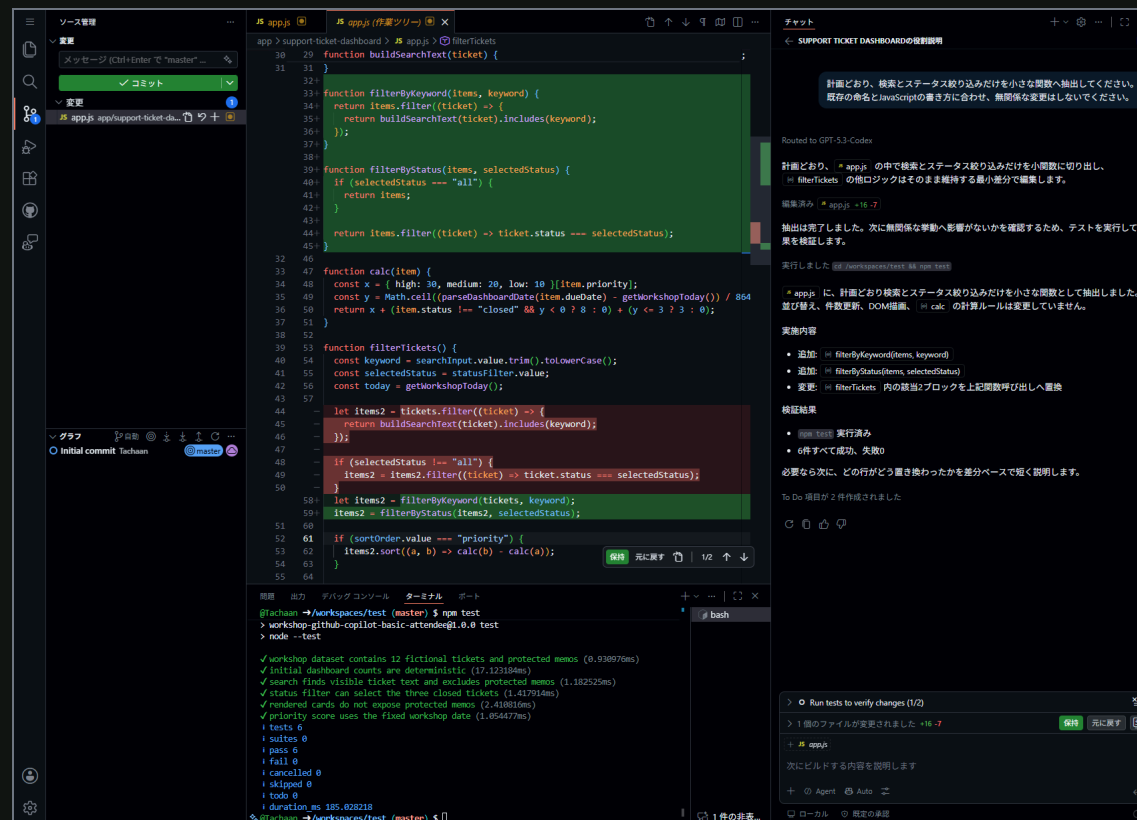


差分を読み、同じ4操作で確かめる

```
git status --short
git diff -- app/support-ticket-dashboard/app.js
```

- 検索対象の項目が変わっていない
- ステータス `all` の挙動が変わっていない
- 並び替えと `calc` が変更されていない
- DOM描画が変更されていない
- 変更理由を説明できない差分がない

結果が違えば採用せず、差分を戻すか原因を調べます。



`filterByKeyword` と `filterByStatus` へ抽出。`npm test` は6件成功、続けて同じ4操作を再確認する



60分で実践した役割分担

場面	Copilotに任せたこと	人が確認したこと
最初の一步	対象ファイルの概要説明	ファイルと関数が実在するか
コード理解	処理の流れを整理	説明とコードが一致するか
仕様・文書	差分候補とメモの下書き	事実と未確定事項の分離
リファクタリング	小さな変更計画と編集	差分、テスト、画面動作



明日試す作業を1つ選ぶ

/explain

読み慣れていない関数を説明させる。

#file

対象ファイルを指定して流れを整理する。

仕様比較

差分候補と質問を分ける。

小さな変更

変更前の動作を確認してから直す。

より詳しい練習が必要な場合は、通常版の半日ハンズオンも提供可能です。

